PARALLEL COMPUTING · TERM PROJECT

Ranking Thai Wikipedia with PageRank on the GPU

188,430 articles · 11.1 million links · sequential C vs three CUDA designs

Trai Pringsulaka

Kasetsart University · November 2026

THE QUESTION

Which Thai Wikipedia articles matter most?

The random reader

A reader opens an article. 85% of the time they click a link on the page; 15% of the time they jump to a random article.

PageRank = the share of time they spend on each article. A page is important if important pages link to it.

Why this is a good GPU problem

- Not a lab problem: an irregular graph, not an array
- Memory access depends on the data (who links to whom)
- Very uneven work: one page has 59,291 incoming links, the median page has 13
- Same computation repeated until it converges

DATASET

The Thai Wikipedia link graph

188,430
articles (vertices)

11.1 M
links (edges), 59 per article

59,291
links into the top page

338
dangling pages (no out-links)

From SQL dump to graph (prep/convert.py)

1. Read 4 SQL tables: page, redirect, linktarget, pagelinks

2. Keep articles: 418,453 pages – 230,023 redirects

3. Point links at redirect targets; drop duplicates and self-links

4. Write CSR of incoming links: row_ptr, col_idx, out_deg

Source: dumps.wikimedia.org/thwiki, dump of 1 October 2026. Subgraphs for scaling: random 10 / 25 / 50 % of articles (seed 42).

THE ALGORITHM

PageRank by power iteration

$$new[v] = (1 - d)/V + d \cdot \left(\Sigma_{u \to v} \, rank[u] / outdeg[u] + D/V \right)$$

d = 0.85 · D = total rank of dangling pages · start with rank = 1/V

Jump term

(1 – d)/V: every page gets the same share from random jumps.

Link term

Each page splits its rank evenly over its out-links; v collects from every page that links to it.

Dangling term

Pages with no out-links would lose rank, so D is spread over all V pages.

Repeat until $\Sigma \, |new - rank| < 10^{-5}$ — the full graph converges in **32 iterations**.

BEST SEQUENTIAL SOLUTION

O(V + E) per iteration, and nothing can do less

```
for u in 0..V-1: // map + reduce
contrib[u] = rank[u]/outdeg[u]
if outdeg[u]==0: D += rank[u]
for v in 0..V-1: // pull
sum = Σ contrib[col_idx[j]],
j in row_ptr[v]..row_ptr[v+1]
next[v] = base + d·D/V + d·sum
diff += |next[v] - rank[v]|
swap(rank, next)
```

Lower bound: every link changes some page's rank, so any correct algorithm must read all E links each iteration. Ω(V + E) = O(V + E).

Why this version is best

- **Incoming CSR (pull):** each next[v] written once, in order
- **contrib precomputed:** V divisions instead of E (59× fewer)
- **Pointer swap:** no copying of rank arrays

Rejected alternatives

- Dense matrix: $V^2 \approx 3.6 \times 10^{10}$ cells
- Linked adjacency lists: pointer chasing, cache misses

PARALLEL DESIGN

One iteration = four building blocks

#	Step	Block	Work W	Steps S
1	contrib[u] = rank[u] / outdeg[u]	map	V	1
2	D = Σ rank of dangling pages	reduce +	V	log V
3	sum[v] = Σ contrib over in-links	gather per page	E	Δin (max in-degree)
4	diff = Σ |next − rank|	reduce +	V	log V

Work-optimal

W = O(V + E) per iteration, the same as the best sequential version, so no work is wasted.

But the span depends on the data

S = O(Δin + log V). On this graph Δin = 59,291, not log V ≈ 18: one page dominates.

VERSION A · PULL

One thread per page

```
__global__ void pullKernel(row_ptr, col_idx,
contrib, rank, next, dangling, diff, V) {
int v = blockIdx.x * blockDim.x + threadIdx.x;
double change = 0.0;
if (v < V) {
float sum = 0.0f;
for (int j = row_ptr[v]; j < row_ptr[v+1]; j++)
sum += contrib[col_idx[j]];
float r = base + DAMPING * sum;
next[v] = r; // only thread v writes here
change = fabsf(r - rank[v]);
}
block_sum_atomic(change, diff); // step 4
}
```

No races, no atomics

Each thread writes only its own next[v]. Pull turns a scatter into a gather.

Graph stays on the GPU

Copied once (≈ 46 MB). Per iteration only 8 bytes come back: the change, for the stop test.

Same result as the CPU

Each sum is added in the same order as the CPU loop: max error 0.

LOAD IMBALANCE

One page can hold up the whole iteration

13
median in-links

563
99th percentile

59,291
maximum (ภาษาอังกฤษ)

In version A that one thread loops 59,291 times while its warp waits. Pull takes 2.6 ms per iteration on the full graph vs 0.47 ms at 50%.

A · PULL

1 thread per page

Simple, no atomics. Span = the longest in-list.

B · WARP PER PAGE

32 threads share a list

Lanes read neighbouring col_idx (coalesced), then a shuffle tree sums the 32 parts. Span ≈ Δin/32 + 5.

C · PUSH

1 thread per link

Perfectly even work, but atomicAdd on popular pages collides: the histogram problem.

ONE GPU ITERATION

Kernels, reductions and what crosses PCIe

`contribKernel`

map rank/outdeg + reduce dangling D

→

`pull | warp | push`

gather new ranks + reduce the change

→

`cudaMemcpy 8 B`

change → CPU; stop if $< 10^{-5}$, else swap

Reduction, as in the lectures

- Tree in shared memory, active threads kept contiguous: no divergence
- One atomicAdd per block, not per thread: privatization
- Partial sums in double, so 188k small ranks do not lose precision

Data movement

- Graph (≈ 46 MB) copied to the GPU once, before iteration 1
- Ranks copied back once, after the last iteration
- Timed both ways: kernels only, and including all copies

CORRECTNESS

GPU matches CPU on every page

Version	Max rel. error	Top 20
A · pull	0	same
B · warp/page	1.4×10^{-5}	same
C · push	6.9×10^{-6}	same

Float addition is not associative, so a different order gives slightly different sums. All three versions converge in the same 32 iterations. Pass rule: error < 10^{-3} and the same top 20, where only exact ties (#17/#18) may swap.

check: PASS

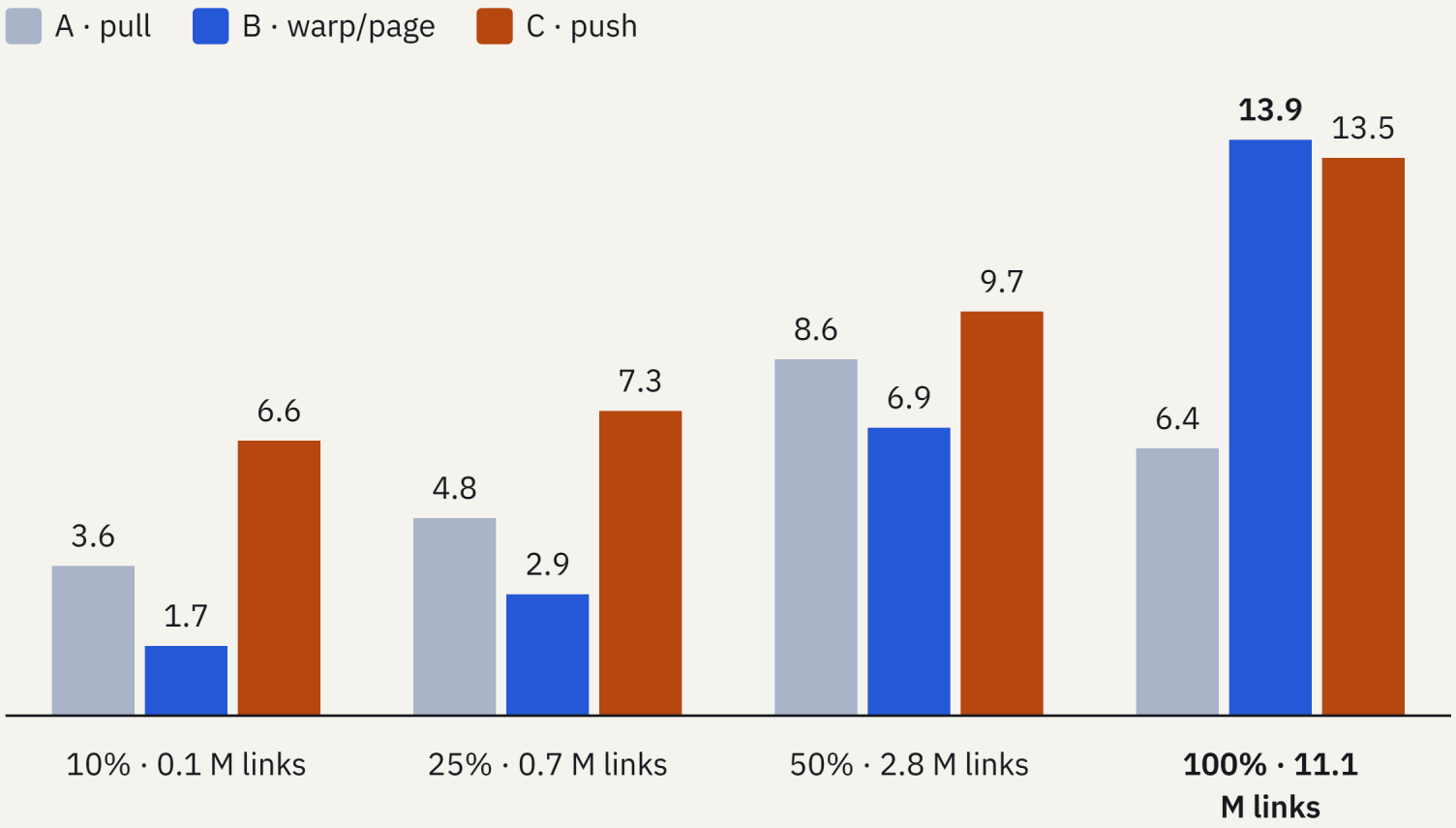
Top 10 Thai Wikipedia articles

#	Article	PageRank
1	ภาษาอังกฤษ	0.01395
2	เวย์แบ็กแมชชีน	0.00668
3	เลขมาตรฐานสากลประจำหนังสือ	0.00494
4	ตัวระบุวัตถุดิจิทัล	0.00237
5	สหรัฐอเมริกา	0.00207
6	ประเทศไทย	0.00177
7	พิกัดภูมิศาสตร์	0.00176
8	เลขมาตรฐานสากลสำหรับนิตยสาร	0.00155
9	ประเทศญี่ปุ่น	0.00120
10	สหราชอาณาจักร	0.00111

Wayback Machine, ISBN, DOI and ISSN probably rank high because citation templates link to them from many articles.

RESULTS · SPEEDUP

13.9× faster on the full graph



Speedup = CPU time ÷ GPU kernel time, both doing exactly 50 iterations. Median of 3 runs.

Speedup **grows with data size**: small graphs cannot fill the GPU, and the fixed cost of 2–3 kernel launches per iteration dominates.

Including all copies, B is still **11.6×** faster.

Google Colab · Tesla T4 (sm_75, 15 GB) · CUDA 13.0 · nvcc -O3 · CPU: one core of the Colab host, same binary

RESULTS · WHICH DESIGN WINS

The best design depends on the graph

Graph	Max in-deg.	CPU ms	A pull	B warp	C push	Fastest
10%	1,155	16.6	4.6	10.0	2.5	C
25%	3,981	60.8	12.8	21.0	8.3	C
50%	8,390	201.5	23.4	29.0	20.7	C
100%	59,291	822.1	127.8	59.1	61.0	B ≈ C

B loses on small graphs

Median in-degree is 1–7, so most of the 32 lanes in a warp have nothing to read.

C wins until hubs appear

One thread per link is perfectly balanced; collisions are rare while no page is huge.

A breaks at 100%

Max in-degree jumps 7× (8,390 → 59,291); A's time per iteration jumps 5.5×.

WHY NOT 100×?

PageRank is limited by memory, not arithmetic

8 B

read per link (col_idx + contrib) for 1 addition

80 GB/s

effective bandwidth of B on the full graph (CPU: 5.7 GB/s)

25%

of the T4's 320 GB/s peak

Where the rest goes

- contrib[col_idx[j]] is a scattered read: neighbouring links point to unrelated pages, so reads are not coalesced
- Each of the 50 iterations pays 2–3 kernel launches, 2–3 memsets and one 8-byte copy
- Copies: the graph crosses PCIe once (≈ 46 MB); C also builds a link-target array on the CPU first

Bytes estimated as 50 × (8E + 28V) for all reads and writes of one iteration.

CONCLUSION

What the project shows

Optimal sequential baseline

O(V + E) per iteration, which matches the lower bound; pull over incoming CSR.

Work-optimal parallel design

map + reduce + gather + reduce; the graph stays on the GPU; 8 bytes back per iteration.

13.9× on real data

11.6× including copies; correct to 1.4×10^{-5} with the same top 20.

The data shape decides

Hubs break one-thread-per-page; warp-per-page or push fixes it.

Next steps

Choose per page: warp for hubs, thread for small pages · renumber pages so linked pages sit close in memory · run on larger wikis.